**UNIT 06 • TI.ST.06.2026**

# MENGIMPLEMENTASIKAN PEMROGRAMAN TERSTRUKTUR

| | | | |
|---|---|---|---|
| **Pengembang** | Aditya Ristanto Mukti | **Proyek** | Website Resmi & Profil Industri PT Global Philsonic Technologies |
| **Deskripsi Unit** | Dokumentasi algoritma pemrograman terstruktur mencakup penguasaan tipe data, percabangan kondisional dengan pola Guard Clauses pada autentikasi, iterasi fungsional deklaratif (filter, map) pencarian produk, prosedur asinkron non-blocking, serta operasi File I/O untuk pembersihan berkas fisik disk server. | | |

## I. MATRIKS PEMENUHAN SUB-TITLE OUTLINE

| No | Sub-Title Outline | Berkas Implementasi | Deskripsi Teknis Hasil Pengerjaan |
|---|---|---|---|
| 1 | **Penguasaan tipe data** | `backend validators & data sanitization` | Validasi ketat tipe data masukan (string, number, boolean, array, object JSON) pada controller. |
| 2 | **Percabangan** | `backend/src/middlewares/authMiddleware.js` | Penerapan Guard Clauses (early return) untuk memangkas nesting kondisional if-else berlebihan. |
| 3 | **Perulangan & Prosedur/Fungsi** | `frontend/src/pages/public/ProductsPage.jsx` | Iterasi deklaratif Array.filter() dan Array.map() untuk pemfilteran multi-kriteria produk. |
| 4 | **Array & File I/O** | `backend/src/utils/fileCleanup.js` | Pembersihan berkas fisik gambar tak terpakai dari storage disk dengan proteksi anti directory-traversal. |

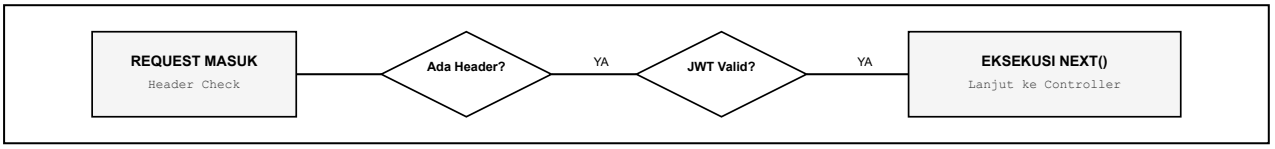
## II. PEMBAHASAN TEKNIS INTI & BUKTI KONKRET

### 6.1 & 6.2. Validasi Tipe Data & Pola Percabangan Guard Clauses

**Lokasi Berkas:** `backend/src/middlewares/authMiddleware.js`

Pola Guard Clause memastikan validasi parameter token dilakukan secara berurutan dan menghentikan eksekusi sedini mungkin jika kondisi prasyarat tidak terpenuhi:

**DIAGRAM ALUR / ARSITEKTUR:**




**Bukti Implementasi Kode Sumber (Visual Studio Code):**

backend/src/middlewares/authMiddleware.js — VS CODE • UTF-8

```
// Penerapan Guard Clauses: Eliminasi Nested If Berlebihan pada Verifikasi Token
const verifyToken = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return error(res, 'Akses ditolak. Token autentikasi tidak ditemukan.', 401);
    }

    const token = authHeader.split(' ')[1];
    if (!token) {
      return error(res, 'Token tidak valid.', 401);
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'fallback_secret_key_change_in_production');

    // Verifikasi ketersediaan user di database
    const [rows] = await pool.execute(
      'SELECT id, name, username, email, role, status FROM users WHERE id = ?',
      [decoded.id]
    );

    if (rows.length === 0) {
      return error(res, 'User pemilik token tidak lagi ditemukan.', 401);
    }

    const user = rows[0];
    if (user.status !== 'active') {
      return error(res, 'Akun Anda dalam status nonaktif. Hubungi Administrator.', 403);
    }

    req.user = user;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return error(res, 'Sesi login telah kadaluarsa. Silakan login kembali.', 401);
    }
    return error(res, 'Token autentikasi tidak valid atau telah dirusak.', 401);
  }
};
```

### 6.3, 6.4 & 6.5. Iterasi Array Deklaratif & Pemfilteran Multi-Kriteria Katalog Produk

**Lokasi Berkas:** `frontend/src/pages/public/ProductsPage.jsx`

Operasi array immutable dilakukan menggunakan metode `.filter()` dan `.some()` untuk menyaring katalog produk manufaktur berdasarkan kategori aktif, kata kunci pencarian SKU, dan spesifikasi teknis:

**Bukti Implementasi Kode Sumber (Visual Studio Code):**

frontend/src/pages/public/ProductsPage.jsx — VS CODE • UTF-8

```
// Filter Array Produk Manufaktur Secara Deklaratif Tanpa Mutasi
const filteredProducts = useMemo(() => {
  return allProducts.filter((prod) => {
    const matchCat = activeCategory === 'all' || prod.category === activeCategory;
    if (!matchCat) return false;

    if (!searchQuery.trim()) return true;
    const q = searchQuery.toLowerCase();
    const title = (prod.title[isId ? 'id' : 'en'] || prod.title.en).toLowerCase();
    const code = prod.modelCode.toLowerCase();
    const desc = (prod.shortDesc[isId ? 'id' : 'en'] || prod.shortDesc.en).toLowerCase();
    const specsMatch = (prod.specs || []).some(
      (s) => s.label.toLowerCase().includes(q) || s.value.toLowerCase().includes(q)
    );

    return title.includes(q) || code.includes(q) || desc.includes(q) || specsMatch;
  });
}, [activeCategory, searchQuery, isId, allProducts]);
```

### 6.6. Operasi File I/O & Pembersihan Berkas Fisik Disk Storage

**Lokasi Berkas:** `backend/src/utils/fileCleanup.js`

Saat produk atau foto fasilitas diperbarui/dihapus dari sistem, fungsi `deleteFileIfLocal` mengeksekusi operasi File I/O dengan validasi keamanan anti directory-traversal untuk menghapus file fisik:

**Bukti Implementasi Kode Sumber (Visual Studio Code):**

backend/src/utils/fileCleanup.js — VS CODE • UTF-8

```
const fs = require('fs');
const path = require('path');

const uploadDir = path.join(__dirname, '../../../uploads');

/**
 * Pembersihan Berkas Fisik: Menghapus File Gambar Produk dari Storage Disk
 */
const deleteFileIfLocal = (filePath) => {
  if (!filePath || typeof filePath !== 'string') return;

  if (filePath.startsWith('/uploads/')) {
    const relativePart = filePath.replace(/^\/uploads\//, '');
    const absolutePath = path.resolve(uploadDir, relativePart);

    // Proteksi keamanan: pastikan path berada strictly di dalam uploadDir
    if (!absolutePath.startsWith(path.resolve(uploadDir))) {
      console.warn(`[FILE CLEANUP BLOCKED] Attempted traversal attack: ${filePath}`);
      return;
    }

    if (fs.existsSync(absolutePath)) {
      try {
        fs.unlinkSync(absolutePath);
        console.log(`[FILE CLEANUP] Successfully deleted unused asset: ${absolutePath}`);
      } catch (err) {
        console.error(`[FILE CLEANUP ERROR] Failed to delete ${absolutePath}:`, err.message);
      }
    }
  }
};

module.exports = { deleteFileIfLocal };
```

**KESIMPULAN KETERCAPAIAN UNIT 06**

Seluruh sub-judul pada unit 'Mengimplementasikan Pemrograman Terstruktur' telah terpenuhi dengan penguasaan tipe data, guard clauses, manipulasi array fungsional, dan pembersihan file I/O aman.